

Cycle Time Boundaries

AA1 – A cycle time cannot be read without its two boundaries – and either can be skipped

What is it?

The report showed a cycle time and never said where the clock runs. A reader had no way of knowing what “Flow Time” even referred to. Daniel Vacanti defines cycle time through two explicitly set boundaries (“Actionable Agile Metrics for Predictability”, 2015, ch. 6). Our report never named them. Now a line above the statistics does – BEFORE the numbers, because without it the numbers mean nothing.

Why it matters

It costs no new input and no new computation. It only surfaces something the tool has known all along and kept to itself. That is exactly where conversations about cycle time break down: two people read the same number and mean different clocks.

The two methods declare differently

- Method A: first entry into the stage to last entry into the stage
- Method B: dwell time summed over all stages before the stage – NO start boundary. Printing a start stage there would assert a boundary that is not in use
- Without --workflow: “boundary not declared” – not a guessed stage

Derived boundaries are counted out

A status can be skipped in either direction. The stage need not be the last one – with Done or Monitoring behind it, an item can jump past the closing status as well. `transform_data` then quietly derives the missing date from a neighbouring stage, and the clock runs between different points than the header announces. The report counts both cases and says so: “43 of 64 items never entered ‘Analysis’, 5 of 64 items never entered ‘Releasing’”. The clean case is stated just as plainly. Not an edge case: in the ART_E test data it is 45 of 78 at the start and 5 at the end.

How to use it

```
python -m build_reports IssueTimes.xlsx --workflow workflow.txt --transitions Transitions.xlsx --browser
```

Nothing to switch on – the line is always there. --workflow supplies the boundaries, --transitions the derived-start check. Without either one, the header says what is missing.